Evaluating Large Language Model Performance on Deterministic Programming Tasks: Accuracy and Prompt-Conditioned Stability

Yongxi Zhou
Northeastern University
Massachusetts, USA
zhou.yongx@northeastern.edu

Abstract

We study how large language models (LLMs) behave under repeated sampling on deterministic programming tasks. Standard code-generation evaluations usually emphasize single-run accuracy or eventual success under repeated sampling, but many deployment settings also require *stability*: similar outcomes across repeated invocations under the same task description and controlled decoding settings. We present a repeated-run evaluation protocol and report complementary metrics for run-level accuracy, first-pass accuracy, retry-free coverage, and problem-level variability. On a recency-based benchmark of 100 LeetCode-style problems, we evaluate 13 models from five provider families under two prompt templates with five repeated runs per problem, yielding 13,000 evaluation instances. Across all configurations, run-level pass rate and perfect stability rate are strongly correlated ($r = 0.985$), but pass rate consistently exceeds retry-free coverage, with an observed gap of 2.4 to 16.2 percentage points. The gap is largest for mid-performing systems, where problem-level outcomes are most variable across runs, and prompt effects are model-dependent rather than uniformly beneficial. These results suggest that repeated-run reliability is a useful complement to conventional accuracy reporting for deterministic code-generation tasks.

1 Introduction

Large language models are increasingly used as *program synthesis* and *code assistance* systems for tasks with deterministic specifications: given a fixed problem statement and test suite, correctness is binary. Conventional benchmarks typically report single-run performance, or summarize repeated sampling through eventual-success metrics such as pass@k (Chen et al., 2021; Austin et al., 2021). In practice, however, repeated calls under the same task description can yield different programs and different pass/fail outcomes, even when the downstream workflow expects reproducible behavior.

This paper studies that repeated-invocation variability directly. Rather than treating repeated sampling only as an opportunity to recover one success among many attempts, we evaluate how much of a model's measured accuracy translates into *retry-free* behavior under fixed conditions. Our focus is therefore complementary to pass@k: we ask not only whether a model can solve a task eventually, but also how reliably it succeeds when invoked multiple times in the same way.

The paper is organized around three empirical questions:

1. How large is the gap between run-level success and retry-free problem coverage under repeated evaluation?

2. How much do prompt choices change repeated-run stability across models?

3. Which aggregate summaries best expose problem-level variability that single-run reporting hides?

We focus on deterministic tasks because they represent a common usage mode for code-generation systems: unit-tested functions, competitive-programming style problems, and API-level transformations with verifiable outputs. In such settings, the accuracy–stability gap has direct practical consequences: a model with 60% run-level pass rate but only 47% perfect stability solves many individual attempts, yet still leaves a substantial fraction of tasks in a retry-required regime. Our aim is therefore not merely to rank models by mean performance, but to characterize how much of their observed accuracy translates into reliable repeated-use behavior.

2 Related Work

Evaluations of LLMs for code generation are commonly framed around pass@k and functional correctness under automated testing (Chen et al., 2021; Austin et al., 2021). These metrics are well suited to questions of eventual success or sampling efficiency: repeated sampling is used to estimate whether at least one candidate among multiple generations is correct. Related prompting work, including zero-shot reasoning and self-consistency, likewise treats multiple samples as a mechanism for improving downstream task performance rather than as an object of measurement in its own right (Kojima et al., 2022; Wang et al., 2022). In contrast, our focus is on what repeated sampling reveals about the stability of outcomes under fixed conditions.

Recent work has also emphasized that code-generation evaluation is sensitive to benchmark construction, execution methodology, and reporting choices (Liu et al., 2024; Du et al., 2024). Our framing is most aligned with this robustness-oriented perspective, but shifts the emphasis from benchmark validity alone to repeated-invocation reliability under a fixed judge. Instead of asking only whether a benchmark is sound or whether a model can eventually solve a task, we ask how often correctness is reproducible at the problem level when the task, prompt family, and decoding regime are held fixed.

More broadly, work on calibration and predictive reliability (Guo et al., 2017) motivates looking beyond average accuracy when systems are used in settings where consistency matters. We adapt that intuition to deterministic program evaluation by measuring not only run-level success, but also retry-free task coverage and variability across repeated runs. This perspective is deliberately narrower than end-to-end agent evaluation: it isolates repeated-run behavior for a single-turn coding setting, while real deployments may involve additional orchestration, tool use, and runtime dependencies whose reliability and security properties must be analyzed separately.

3 Experimental Setup

3.1 Problem Dataset

We evaluate on a curated dataset of deterministic programming problems, denoted $\mathcal{D} = \{(x_i, T_i)\}_{i=1}^{N}$, where x_i is the natural-language specification (including I/O format and constraints) and T_i is a deterministic test harness. Each problem is graded by executing the model-produced program against T_i, producing a binary correctness indicator. These tasks are instantiated as LeetCode problems with platform-provided acceptance criteria, and all generations are evaluated as Python solutions against Python-based submission templates.

Problem selection. To reduce the risk of benchmark contamination from model training data, we select the $N = 100$ most recently published problems available on the platform at collection time, sampling the newest problems first. Problems requiring paid-only access are excluded. No further manual curation is applied; the final benchmark is entirely determined by recency rank. This construction does not guarantee that evaluated models have never encountered related problem statements during training, but it substantially reduces the probability of direct memorization compared to historically popular problems.

The dataset spans three difficulty tiers (Easy, Medium, Hard) and covers a broad range of algorithmic topics including Array, Dynamic Programming, String, Graph, and Simulation. All problems satisfy the following properties:

- **Deterministic grading:** each T_i yields a deterministic accept/reject outcome.
- **Self-contained evaluation:** no network access or external dependencies are required at runtime.
- **Fixed snapshot:** all models are evaluated on the identical problem set to ensure comparability across runs.

In the evaluation framework, this benchmark is stored as a fixed local dataset snapshot rather than fetched online at run time. This design ensures that all model families are evaluated against the same problem set and ordering, and reduces the chance that later platform changes silently alter the benchmark during the experimental campaign.

3.2 Models

We evaluate 13 models across five provider families, covering a range of model scales, training objectives, and architectural choices, including both standard and reasoning-specialized variants. The evaluated models are listed in Table 1.

Table 1: Models evaluated in this study, grouped by provider family. † denotes reasoning-specialized models.

Family	Model	Type
OpenAI	GPT-4.1	flagship
	GPT-4.1-mini	lightweight
Anthropic	Claude Sonnet 4.5	flagship
	Claude Haiku 4.5	lightweight
Google	Gemini 3.1 Pro (preview)	flagship
	Gemini 3 Flash (preview)	efficient
	Gemini 3.1 Flash-Lite (preview)	lightweight
Qwen	QwQ-Plus	reasoning†
	Qwen-Plus	flagship
	Qwen-Max	large
	Qwen-Turbo	lightweight
DeepSeek	DeepSeek-R1	reasoning†
	DeepSeek-V3.2	flagship

3.3 Evaluation Framework

For a model m and a fixed evaluation configuration c (prompt template, decoding parameters, toolchain), we run each problem x_i multiple times. Each run produces a candidate program which is executed against T_i.

Let $Y_{i,r}^{(m,c)} \in \{0,1\}$ denote whether run $r \in \{1,\dots,R\}$ passes all tests for problem i. We treat compilation errors, runtime errors, timeouts, and wrong answers as failures (0).

The evaluation pipeline separates generation from execution. For large-scale experiments, model outputs are first generated through provider APIs, cached locally, normalized into a shared format, and only then submitted to the deterministic judge. This design makes experiments resumable across providers and reduces the risk that transient provider-side failures are conflated with downstream execution outcomes.

3.4 Repeated-Run Methodology

We adopt a repeated-run design to estimate both mean performance and variability. For each (m, c) pair we:

1. Sample R independent generations per problem under configuration c.
2. Evaluate each generation deterministically against T_i.
3. Aggregate outcomes into run-level and problem-level metrics with uncertainty estimates.

The primary controlled intervention is the prompt template. We compare two prompt variants while holding the dataset, model, decoding parameters, and execution environment fixed. All models are evaluated with temperature 0.3, top-$p = 0.9$, and a maximum output budget of 4096 tokens, using $R = 5$ repeated runs per problem per prompt configuration. We use a non-zero temperature because the goal of the study is to characterize repeated-run variability under realistic stochastic decoding rather than to collapse outputs toward near-deterministic behavior at $T = 0$. The two prompt templates—DETAILED and MINIMAL—are held fixed across all models to enable a controlled comparison of prompt sensitivity.

Prompt templates. Both prompts instruct the model to return plain Python source code only and to follow the provided LeetCode method signature exactly. The DETAILED prompt uses a longer structured template emphasizing careful constraint reading, edge-case coverage, submission readiness, and accepted-style performance. The MINIMAL prompt conveys the same task more compactly, with lighter instruction scaffolding. Because both templates share the same output constraints and code template, the prompt comparison isolates instruction density rather than changing the task itself.

Normalization and execution. Raw generations from different providers are normalized into a common local representation before evaluation. Code is extracted from model responses, basic syntax validity is checked, and each candidate is then submitted to the same deterministic judge. This normalization step is important because provider APIs differ in output packaging, token accounting, and batch interfaces even when the downstream task is identical.

3.5 Metrics Definition

Run-Level Pass Rate (RLPR).

$$\mathrm{RLPR}(m,c) = \frac{1}{NR}\sum_{i=1}^{N}\sum_{r=1}^{R} Y_{i,r}^{(m,c)}.$$

RLPR measures the probability that a *random invocation* succeeds.

First-Pass Accuracy (FPA).

$$\mathrm{FPA}(m,c) = \frac{1}{N}\sum_{i=1}^{N} Y_{i,1}^{(m,c)}.$$

FPA matches common single-run benchmark reporting and enables comparability to existing results.

Perfect Stability Rate (PSR).

$$\mathrm{PSR}(m,c) = \frac{1}{N}\sum_{i=1}^{N} \mathbf{1}\left[\sum_{r=1}^{R} Y_{i,r}^{(m,c)} = R\right].$$

PSR quantifies the fraction of tasks for which the system is reliably correct without retries.

Average Variance (AV).

$$\mathrm{AV}(m,c) = \frac{1}{N}\sum_{i=1}^{N} \frac{R}{R-1}\hat{p}_i^{(m,c)}\left(1-\hat{p}_i^{(m,c)}\right),$$

where $\hat{p}_i = \frac{1}{R}\sum_r Y_{i,r}$. This finite-sample correction yields the unbiased Bernoulli variance estimate for each problem under repeated sampling. AV captures average instability across tasks.

95% Confidence Intervals. For RLPR, FPA, and PSR, we compute 95% Wilson score intervals. For AV, we use nonparametric bootstrap over problems.

Prompt comparison tests. Because DETAILED and MINIMAL prompting are evaluated on the same underlying problems, prompt comparisons are paired rather than independent. For PSR, we therefore compare prompt conditions at the problem level using an exact McNemar test on the binary indicator of whether all $R = 5$ runs for a problem pass under each prompt condition. We use these tests as a conservative check on whether observed prompt differences are larger than expected from the finite benchmark size alone.

Scope of inference. Unless otherwise stated, reported statistics are descriptive summaries of the observed benchmark and evaluation protocol rather than claims of universal model behavior. In particular, prompt comparisons should be interpreted as controlled comparisons between the two specific templates used in this study, not as claims about prompt engineering in general.

4 Results

4.1 Overall Accuracy and Stability

Table 2 reports RLPR, FPA, PSR, and AV for all 26 model/prompt configurations. The top performers under RLPR are Gemini 3.1 Pro (86.2% D / 85.8% M), DeepSeek-R1 (84.2% D / 84.4% M), and Gemini 3 Flash (80.8% D / 83.2% M). More broadly, the table shows that high run-level performance and high retry-free coverage often co-occur, but not in a one-to-one way: models with similar RLPR can still differ materially in PSR and AV.

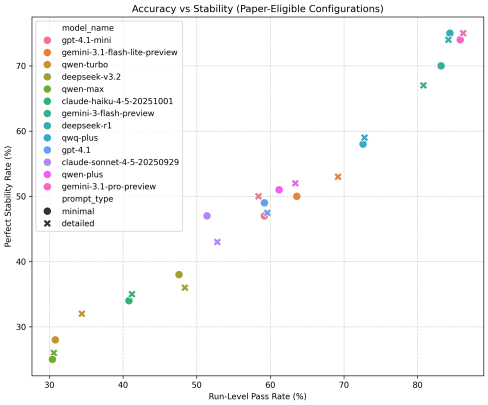

Figure 1: RLPR vs. PSR scatter plot across all 26 configurations (13 models × 2 prompt types). Points are colored by model family and shaped by prompt type. The diagonal marks PSR = RLPR; points below indicate configurations where mean success overstates stable problem coverage.

Figure 1 plots RLPR against PSR for all 26 configurations. All points fall below the diagonal, confirming that RLPR consistently overstates production reliability. Across all configurations, RLPR and PSR are strongly correlated ($r = 0.985$, $p < 0.001$), which is directionally expected because both metrics are derived from the same underlying per-problem success structure. The informative result is therefore not the existence of a high correlation itself, but the persistent and non-uniform gap between run-level success and retry-free coverage.

The gap $\Delta = \mathrm{RLPR} - \mathrm{PSR}$ is non-uniform and reaches its maximum in the mid-accuracy tier. Gemini 3.1 Flash-Lite has the largest gap (16.2% under DETAILED), followed by Gemini 3 Flash and QwQ-Plus (both 13.8%). At the low end, Qwen-Turbo has a gap of only 2.4%—not because it is uniformly reliable, but because its outcomes are highly polarized: many problems are either always failed or always passed, leaving less room for intermediate trial-to-trial variability. This pattern is reflected in AV: Qwen-Turbo has the lowest AV (0.0088), while Gemini 3.1 Flash-Lite has the highest (0.0528). Together, PSR and AV help distinguish between systems that are consistently strong, consistently weak, and unstable in the middle.

Table 2: Main results across all 13 models and two prompt configurations (100 problems, $R = 5$ runs, 13,000 total evaluation instances). RLPR, FPA, PSR, and AV are shown for both DETAILED (D) and MINIMAL (M) prompts. All values are point estimates in %; corresponding 95% confidence intervals are computed as described in Section 3 and reported in the archived per-model summaries.

Family	Model	RLPR		FPA		PSR		AV	
		D	M	D	M	D	M	D	M
OpenAI	GPT-4.1	59.6	59.2	61.6	56.0	47.5	49.0	4.36	4.08
	GPT-4.1-mini	58.4	59.2	60.0	61.0	50.0	47.0	3.28	4.48
Anthropic	Claude Sonnet 4.5	52.8	51.4	52.0	49.0	43.0	47.0	3.36	1.68
	Claude Haiku 4.5	41.2	40.8	42.0	43.0	35.0	34.0	2.40	2.00
Google	Gemini 3.1 Pro (preview)	**86.2**	85.8	87.0	88.0	**75.0**	74.0	3.36	3.92
	Gemini 3 Flash (preview)	80.8	**83.2**	83.0	83.0	67.0	70.0	5.28	4.40
	Gemini 3.1 Flash-Lite (preview)	69.2	63.6	69.0	65.0	53.0	50.0	5.28	5.44
Qwen	QwQ-Plus	72.8	72.6	75.0	70.0	59.0	58.0	4.80	4.72
	Qwen-Plus	63.4	61.2	64.0	61.0	52.0	51.0	4.72	4.64
	Qwen-Max	30.6	30.4	30.0	30.0	26.0	25.0	2.24	2.24
	Qwen-Turbo	34.4	30.8	33.0	30.0	32.0	28.0	0.88	1.04
DeepSeek	DeepSeek-R1	84.2	**84.4**	83.0	83.0	74.0	**75.0**	3.76	3.20
	DeepSeek-V3.2	48.4	47.6	51.0	47.0	36.0	38.0	5.52	4.00

4.2 Problem-Level Stability Analysis

Figure 2 presents a cross-model stability overview. Each row corresponds to one of the 13 models, ordered by RLPR under DETAILED prompting (highest at top). Each column corresponds to one of the 100 problems, grouped by difficulty tier (Easy, Medium, Hard) and sorted within each tier by decreasing mean pass rate across all models. Cell color encodes the empirical pass probability for that model–problem pair, averaged over $R = 5$ runs: green indicates a problem solved consistently on every run, red indicates consistent failure, and yellow indicates high trial-to-trial variability.

Figure 3 shows the distribution of per-problem pass rates aggregated across all models. Qwen-Turbo shows the most polarized profile (AV $=$ 0.0088), while Gemini 3.1 Flash-Lite has the highest AV (0.0528), indicating substantial trial-to-trial variability.

This view is useful because it exposes qualitatively different reliability profiles that similar aggregate pass rates can hide. A model with many $\hat{p}_i \approx 1$ and $\hat{p}_i \approx 0$ problems behaves differently from a model with many $\hat{p}_i \approx 0.4$ or 0.6 problems, even when their average run-level pass rate is similar.

We further note that retry-based recovery does not eliminate instability. PSR directly measures the fraction of tasks that do not require retries, while AV highlights how much of the remaining workload lies in a stochastic regime.

4.3 Difficulty Gradient and Failure Modes

To characterize where instability occurs, we performed a post-hoc aggregate analysis over the archived trial-level outputs from the completed paper runs. Pooling outcomes by benchmark difficulty reveals a pronounced gradient: run-level pass rates are highest on Easy problems (90.6%), lower on Medium problems (62.2%), and substantially lower on Hard problems (34.0%). This indicates that the overall accuracy–stability pattern is not driven by a small number of outlier tasks; it coexists with a broad increase in failure probability as algorithmic difficulty rises.

Failure modes are also non-uniform. Across the archived outputs, the dominant non-accept verdict is *Wrong Answer*, followed by *Time Limit Exceeded* and *Runtime Error*. This ordering suggests that most failures arise from selecting an incorrect or incomplete algorithm rather than from pure formatting noise alone. At the same time, the prevalence of timeouts and runtime failures increases notice-

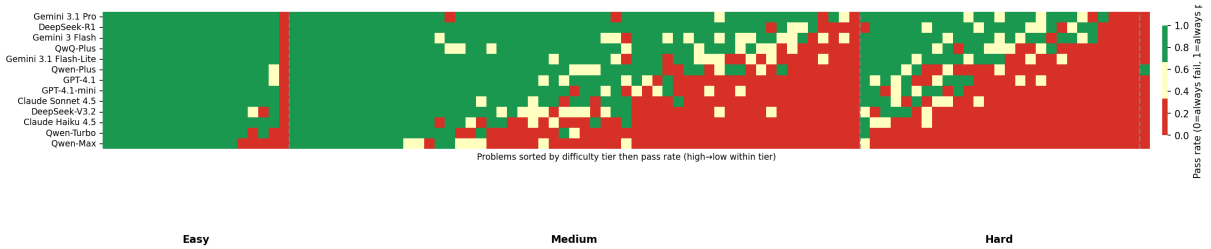

Figure 2: Cross-model stability heatmap (DETAILED prompt). Rows: 13 models sorted by RLPR descending. Columns: 100 problems grouped by difficulty tier with dashed separators, and ordered within each tier by decreasing mean pass rate. Cell color encodes empirical pass rate over $R = 5$ runs (green $= 1.0$, red $= 0.0$, yellow ≈ 0.5).

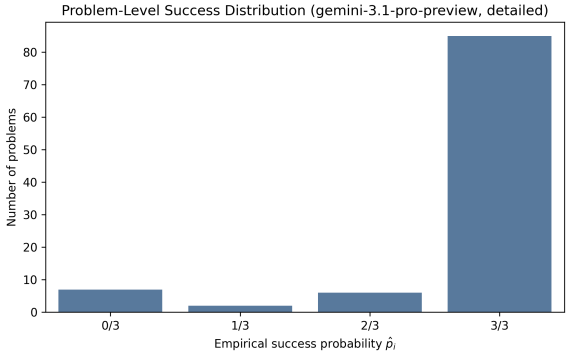

Figure 3: Problem-level success distribution for all 13 models (DETAILED prompt). Each panel shows the fraction of problems in each empirical pass-probability bin $\hat{p}_i \in \{0/5, \ldots, 5/5\}$, arranged in decreasing order of PSR.

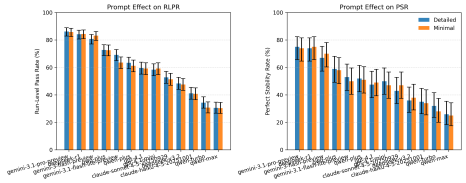

Figure 4: Prompt sensitivity: paired scatter of PSR under DETAILED vs. MINIMAL prompting for all 13 models. Points above the diagonal indicate that detailed prompting improves stability. Points are colored by model family.

ably on Medium and Hard problems, indicating that instability is partly tied to implementation robustness and complexity-sensitive performance rather than only to binary conceptual success or failure.

These observations refine the interpretation of PSR and AV. A low PSR can reflect at least two qualitatively different regimes: algorithmic inconsistency, where runs alternate between accepted and wrong solutions, and implementation fragility, where some runs fail because the produced program is incomplete, inefficient, or execution-unsafe. Distinguishing these regimes is useful for deployment, because they imply different mitigation strategies such as prompt refinement, stricter output constraints, or downstream verification.

4.4 Prompt Sensitivity

Figure 4 visualizes the prompt effect as a paired scatter plot: each of the 13 models appears as a single point with its MINIMAL-prompt PSR on the x-axis and DETAILED-prompt PSR on the y-axis. Points above the diagonal indicate that detailed prompting improves stability.

Prompt effects are model-dependent and non-uniform. Most models show small differences between prompt conditions (within 3–5 percentage points of PSR). However, Claude Sonnet 4.5 shows a reversal, with higher PSR under MINIMAL (47.0%) than DETAILED (43.0%) prompting, while Gemini 3 Flash shows the opposite pattern (67.0% D vs. 70.0% M). In this dataset, prompt sensitivity is therefore observable but heterogeneous, and no single template uniformly dominates across all model families.

To assess whether these prompt differences are statistically compelling at the problem level, we ran exact McNemar tests on the paired PSR indicators for each model. Under this test, no model reaches $p < 0.05$; the largest observed PSR shifts are 4 percentage points (Qwen-Turbo in favor of DETAILED, Claude Sonnet 4.5 in favor of MINIMAL), and both correspond to $p = 0.219$ under exact paired testing. Across the 13 fully evaluated models, 8 favor DETAILED and 5 favor MINIMAL, with a mean absolute PSR difference of approximately 2.1 percentage points. We therefore interpret the prompt results as evidence of model-dependent directional variation, but not as strong evidence that either template yields a consistent advantage at the current benchmark size.

4.5 Accuracy–Stability Gap

Although RLPR and PSR are strongly correlated overall, the gap $\Delta = \mathrm{RLPR} - \mathrm{PSR}$ is practically significant and non-uniform across models. Across all 26 configurations, Δ ranges from 2.4% (Qwen-Turbo, DETAILED) to 16.2% (Gemini 3.1 Flash-Lite, DETAILED). The gap peaks in the middle of the performance spectrum, where models succeed frequently enough to generate trial-to-trial variability but not consistently enough to eliminate it. This non-linearity means that a single RLPR ranking cannot be used to infer the magnitude of the accuracy–stability gap: two models with similar RLPR can differ substantially in how much of their accuracy converts to retry-free reliability. PSR makes this distinction explicit.

5 Scope and Practical Implications

This study is intentionally scoped to repeated-run correctness under a fixed evaluation harness. Stability remains practically important because deterministic programming tasks are used in settings that assume reproducibility. When a system is invoked repeatedly under the same input, instability creates concrete risks: non-reproducibility complicates debugging; workflow overhead arises from retry requirements; and validation burden increases from inconsistent outputs.

From a decision-theoretic perspective, selecting a model based solely on pass rate optimizes expected success but ignores the cost of stochastic failures. PSR provides a direct estimate of the workload fraction that is "retry-free." Even when two models have similar RLPR, they may differ substantially in PSR: this gap represents problems that require repeated invocations to reliably succeed in production, incurring latency, cost, and engineering complexity. A model ranked lower by RLPR could be preferable in deployment if its PSR is higher—a tradeoff invisible to single-run benchmarking.

For this reason, we view repeated-run metrics as complementary rather than replacement metrics. RLPR remains useful for characterizing average invocation success, while PSR and AV summarize whether that success is concentrated in consistently solved problems or dispersed across unstable outcomes.

At the same time, our deployment interpretation remains intentionally narrow. In practical systems, generated code may flow through larger agentic or tool-mediated runtimes, introducing additional operational and security risks beyond single-turn correctness alone (Jiang et al., 2026). Our evaluation does not attempt to measure those broader runtime-supply-chain concerns; instead, it isolates one lower-level component of reliability: whether the model repeatedly produces correct solutions for the same deterministic task.

6 Limitations

Several limitations qualify our results. First, the benchmark is relatively small ($N = 100$) and restricted to recent LeetCode-style problems. This design helps reduce direct overlap with heavily circulated historical benchmarks, but it does not eliminate contamination risk and should not be interpreted as proof of benchmark novelty. Second, our repeated-run protocol is intentionally narrow: we vary prompt template and repeated sampling while holding the evaluation harness fixed. As a result, the study does not characterize the effects of broader prompt engineering, tool use, self-repair, retrieval, test-time selection, or multi-turn interaction.

Third, model access is mediated through provider APIs and platform infrastructure. Even though task grading is deterministic, we do not control all backend implementation details, model refreshes, or service-side changes that may affect outputs over time. Fourth, our summary metrics compress distinct failure modes into a binary pass/fail outcome. This is appropriate for end-task correctness, but it does not by itself explain whether instability arises from reasoning errors, formatting mistakes, runtime failures, or judge-specific edge cases. Finally, our conclusions are limited to Python code generation on deterministic coding tasks with executable tests. They should not be read as a complete account of reliability for other programming languages, for open-ended generation tasks, or for production systems that include additional orchestration and verification layers.

7 Ethics Statement

This paper evaluates commercial and open-access LLM APIs on deterministic programming tasks. The study does not involve human subjects, personal data collection, or annotation labor. The main ethical considerations are instead tied to platform use and deployment interpretation. First, the evaluation relies on automated submission to an external

online judge. Such use should be understood as research benchmarking under the platform's operational constraints, and any public release should avoid exposing credentials, session artifacts, or procedures that would enable abusive automation. Second, the evaluated models are accessed through third-party APIs whose outputs and availability may be governed by provider-specific terms of service and may change over time. Our reported results should therefore be interpreted as measurements of accessed systems under a documented evaluation protocol, not as immutable properties of any provider's model family.

More broadly, stronger performance on coding benchmarks can increase both beneficial and harmful capabilities. Better repeated-run reliability can support legitimate software engineering workflows, but it may also lower the barrier to generating code in settings where security review is weak. For this reason, we frame our contribution as an evaluation methodology rather than as an endorsement of fully autonomous code deployment.

8 Conclusion

We presented a repeated-run evaluation framework for deterministic programming tasks and reported complementary summaries that separate mean performance from stability. Across 13 models, 100 problems, and 5 repeated runs per configuration (13,000 total instances), run-level pass rate and retry-free coverage are strongly correlated, but run-level accuracy consistently exceeds perfect stability. The resulting gap is non-uniform across models and is largest in the middle of the performance spectrum, where problem outcomes are most variable across runs. Prompt effects are also heterogeneous across model families rather than uniformly positive. Taken together, these findings suggest that repeated-run evaluation is a useful complement to standard code-generation reporting whenever outputs are consumed in deterministic, reproducibility-sensitive workflows.

References

Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and 1 others. 2021. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*.

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, and 1 others. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*.

Xueying Du, Mingwei Liu, Kaixin Wang, Hanlin Wang, Junwei Liu, Yixuan Chen, Junjie Feng, Guangyu Zhu, Shang-Wei Li, and Yang Liu. 2024. Evaluating large language models in class-level code generation. *arXiv preprint arXiv:2308.01861*.

Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q Weinberger. 2017. On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning*, pages 1321–1330.

Xiaochong Jiang, Shiqi Yang, Wenting Yang, Yichen Liu, and Cheng Ji. 2026. Agentic ai as a cybersecurity attack surface: Threats, exploits, and defenses in runtime supply chains. *Preprint*, arXiv:2602.19555.

Takeshi Kojima, Shixiang Shane Gu, Machel Reid, Yutaka Matsuo, and Yutaka Iwasawa. 2022. Large language models are zero-shot reasoners. *arXiv preprint arXiv:2205.11916*.

Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. 2024. Is your code generated by chatgpt really correct? rigorous evaluation of large language models for code generation. *Advances in Neural Information Processing Systems*, 36.

Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. 2022. Self-consistency improves chain of thought reasoning in language models. *arXiv preprint arXiv:2203.11171*.